TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A Project Report

On

Pynter: A Raster Graphics Paint Application

Submitted By:

Aswin Kandel (THA081BCT004)

Dikesh Manandhar (THA081BCT008)

Kishan Kumar Shah (THA081BCT014)

Pujag Dallakoti (THA081BCT024)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

Under the Supervision of

Er. Kobid Karkee

March, 2026

ACKNOWLEDGEMENT

We express our sincere thanks to the Institute of Engineering, Thapathali Campus, for giving us the chance to improve our knowledge and abilities through this project. We extend our sincere gratitude to the Department of Electronics and Computer Engineering for their ongoing assistance and for providing us with essential resources and information. We are particularly grateful to our supervisors, Er. Kobid Karkee, along with Er. Rajan Bhandari, for their essential guidance, support, and expert insights throughout this project's duration. Their guidance was vital for the successful fulfillment of our project.

We would also like to convey our gratitude to our colleagues and our supervisors for their support and encouragement. This project was more than just an academic task; it was an experience that enabled us to use our practical skills in Computer Graphics, particularly in Python with the Pygame library. The educational experience and practical application have been incredibly advantageous, and we appreciate everyone who contributed to this learning journey.

Aswin Kandel (THA081BCT004) _____

Dikesh Manandhar (THA081BCT008) _____

Kishan Kumar Shah (THA081BCT014) _____

Pujag Dallakoti (THA081BCT024) _____

Date: March, 2026

ABSTRACT

Pynter is a raster graphics paint application built with Python and Pygame. It implements core computer graphics algorithms from scratch - Bresenham's Line Algorithm, Midpoint Circle and Ellipse Algorithms, Bezier Curves, BFS-based Flood Fill, and 2D geometric transformations - across 18 drawing tools in an interactive editor. Instead of relying on high-level drawing functions, shapes are rasterized directly at the pixel level. The editor also includes an RGB color picker, undo/redo support, and custom bitmap-based UI elements. The project serves as a practical demonstration of how classical scan-conversion and filling techniques can be applied in a real, interactive application. It was developed as a mini project for the Computer Graphics and Visualization course at the Institute of Engineering, Thapathali Campus.

Keywords: Raster Graphics, Scan Conversion, Bresenham's Algorithm, Midpoint Algorithm, Bezier Curves, 2D Transformations, Pygame

Table of Content

ACKNOWLEDGEMENT ..... **I**

ABSTRACT ..... **II**

LIST OF FIGURES ..... **VI**

LIST OF ABBREVIATION ..... **VII**

1. INTRODUCTION **1**

1.1 Background 1

1.2 Motivation 2

1.3 Problem Statement 2

1.4 Objective 3

1.5 Scope and Limitations 3

1.5.1 Scope 3

1.5.2 Limitations 3

2. LITERATURE REVIEW **4**

2.1 Background of Raster Graphics System 4

2.2 Computer Graphics Algorithms 4

2.3 Application of Raster Graphics Editors 5

2.4 Limitations and Challenges 5

2.5 Development Approach 5

3. TOOLS AND TECHNOLOGIES **6**

4. SYSTEM ARCHITECTURE AND DESIGN **7**

4.1 Overview Block Diagram 7

4.2 Component Architecture 8

4.3 Rendering Pipeline 9

5. ALGORITHM IMPLEMENTATION **11**

5.1 Scan Conversion Algorithm 11

5.1.1 Bresenham's Line Algorithm 11

5.1.2 Midpoint Circle Algorithm 12
5.1.3 Midpoint Ellipse Algorithm 12
5.2 Area Filling and Curves 12
5.2.1 Flood Fill Algorithm 12
5.2.2 Bezier Curve Rendering 13
5.3 Transformations and Other Techniques 14
5.3.1 2D Geometric Transformations 14
5.3.2 Mandala / Symmetry Drawing 14
5.3.3 Archimedes Spiral 14
5.3.4 Monte Carlo Sampling (Spray Tool) 15
5.3.5 Linear Interpolation for Opacity Blending (Eraser) 15
6. WORKING PRINCIPLE 16
6.1 Setup 16
6.1.1 Initialization 16
6.1.2 Tool Selection 16
6.1.3 Color Selection 16
6.2 Drawing Pipeline 17
6.2.1 Drawing Operation 17
6.2.2 Preview Rendering (Rubber-Band Effect) 17
6.3 State Management 18
6.3.1 Undo and Redo System 18
6.3.2 Saving the Canvas 18
6.3.3 Continuous Rendering Loop 18
7. IMPLEMENTATION DETAILS 19
7.1 Features and Tools 19
7.2 Adjustable Tool Settings 20
7.3 1-Bit Bitmap Icon Rasterization 20

8. USER INTERFACE DESIGN 21

8.1 Color Scheme 21

8.2 Layout Structure 21

8.3 1-Bit Bitmap Icons 21

8.4 Key Controls 22

9. RESULT AND ANALYSIS 23

9.1 Initial Window 23

9.2 Color Select 23

9.3 Drawing Demo 24

10. FUTURE ENHANCEMENTS 25

11. CONCLUSION 26

12. APPENDICES 27

REFERENCES 28

List of Figures

Fig 4-1: Overview Block Diagram 8
Fig 4-3: Rendering Pipeline Flowchart 10
Fig 9-1: Main Application Window 23
Fig 9-2: Color Select/Picker 23
Fig 9-3: Drawing Demo [1] 24
Fig 9-3: Drawing Demo [2] 24
Fig 12-1 : Gantt Chart 27

List of Abbreviation

2D	Two-Dimensional
ABC	Abstract Base Class
BFS	Breadth-First Search
BMP	Bitmap Image File
CAD	Computer-Aided Design
DDA	Digital Differential Analyzer
FPS	Frames Per Second
GPU	Graphics Processing Unit
GUI	Graphical User Interface
IOE	Institute of Engineering
JPEG	Joint Photographic Experts Group
PNG	Portable Network Graphics
RGB	Red, Green, Blue
UI	User Interface

1. INTRODUCTION

In this project, we developed a raster graphics editor called Pynter (from "Python" + "Painter") using Python and the Pygame library. The application implements core computer graphics algorithms from scratch, demonstrating the practical side of the Computer Graphics and Visualization course. Pynter allows users to create digital artwork using multiple drawing tools based on classical scan conversion algorithms, geometric transformations, and parametric curve techniques.

Instead of using high-level drawing functions like most graphics' apps do, this project implements the core algorithms from scratch. Lines, circles, ellipses, and curves are rasterized directly on a pixel grid using the algorithms we studied in the course. This gives us a hands-on understanding of how continuous mathematical shapes get converted into discrete pixels on display, which is basically bridging the gap between graphics theory and actual application development.

1.1 Background

Computer graphics applications rely on algorithms that convert mathematical descriptions of shapes into raster images displayed on screen. In this project, Python and the Pygame graphics library [2] are used to implement these algorithms in a fully functional drawing environment. The key algorithms and techniques implemented are:

- Bresenham's Line Algorithm, Midpoint Circle Algorithm, and Midpoint Ellipse Algorithm for rasterizing geometric primitives
- BFS-based Flood Fill for area filling
- Bezier curves evaluated through Bernstein polynomials for parametric curve generation
- Four basic 2D geometric transformations: translation, rotation, scaling, and reflection
- These transformations are used in interactive tools like object selection/movement, symmetry drawing, and magnification
- Monte Carlo sampling for a spray-painting tool

1.2 Motivation

Modern digital drawing tools such as Adobe Photoshop, GIMP, and Microsoft Paint have significantly transformed the way images are created. Open-source alternatives like Krita and Pinta have further expanded accessibility to digital art creation, enabling more users to explore and develop creative work.

The motivation for Pynter comes from understanding how these tools function internally. It involves concepts such as smooth line drawing on a pixel grid, rasterization of circles without floating-point arithmetic, and determining region boundaries during fill operations. These ideas provide deeper insight into computer graphics. By implementing these algorithms from scratch rather than using built-in graphics functions, a better understanding of scan conversion, geometric transformations, and parametric curve generation is achieved, while also creating a usable system instead of just a demonstration.

1.3 Problem Statement

Making a raster graphics editor that works and shows graphics algorithms in action comes with a few technical challenges:

- Getting the scan conversion algorithms (Bresenham's Line, Midpoint Circle/Ellipse) to produce visually accurate output, since they must approximate smooth shapes on a pixel grid using only integer math
- Making the rendering fast enough to handle drawing, tool switching, and screen updates without feeling laggy
- Getting 2D transformations right, especially rotations and reflections around arbitrary pivot points
- The Bezier curve tool needs to evaluate Bernstein polynomials for multiple control points, which adds some complexity
- Building a decent UI for tool selection, color picking, and undo/redo since Pygame does not have built-in GUI widgets.

1.4 Objective

- To implement core computer graphics algorithms such as Bresenham's Line Algorithm, Midpoint Circle Algorithm, Midpoint Ellipse Algorithm, and Flood Fill for rasterizing geometric primitives
- To develop a fully interactive raster graphics editor that enables users to create digital drawings using multiple tools, color selection, and undo/redo functionality

1.5 Scope and Limitations

1.5.1 Scope

- Implement geometric primitives (lines, circles, ellipses, rectangles, curves) using classical scan conversion algorithms developed from scratch
- Includes drawing tools: pencil, brush, eraser, line, rectangle, circle, ellipse, triangle, fill, spray, symmetry/mandala, Bezier curve, and more (18 total)
- Supports real-time rendering with responsive user interaction
- Users can select colors from 24 preset swatches or use an RGB color picker
- Snapshot-based undo/redo mechanism (up to 50 steps)
- Implemented in Python using Pygame; runs on Windows, macOS, and Linux

1.5.2 Limitations

- Focuses exclusively on 2D graphics - no 3D transformations, hidden surface removal, or lighting models
- No anti-aliasing, layered editing, or complex image manipulation
- Single canvas layer: images can be saved as PNG but importing external images is not supported
- These limitations are intentional - they keep the project focused on the core algorithms covered in the course

2. LITERATURE REVIEW

Pynter is built on core concepts from computer graphics - raster graphics systems, scan conversion algorithms, geometric transformations, and interactive graphics programming.

2.1 Background of Raster Graphics System

Raster graphics systems store images as a grid of pixels in a frame buffer, where each pixel has RGB color values. This pixel-based approach is how most modern graphics applications work. Software like Adobe Photoshop, GIMP, and Microsoft Paint shows how raster techniques let users create and edit digital images. Open-source tools like Krita, Pinta, and Aseprite have made digital drawing even more accessible, all relying on rasterization to convert continuous shapes into discrete pixels. Pyint Pixel-Painter [3], Pixel-Craft [4], and a basic paint program shared on Reddit [5] are some open-source Pygame-based paint applications that served as early references.

2.2 Computer Graphics Algorithms

Efficient scan conversion algorithms are at the heart of raster graphics. The key algorithms relevant to this project are:

- Bresenham's Line Algorithm [6] - introduced in 1965, draws lines using only integer arithmetic
- DDA (Digital Differential Analyzer) - another common line drawing method covered in the textbook [1], though Bresenham's is generally preferred for efficiency
- Midpoint Circle and Midpoint Ellipse algorithms - extend the midpoint idea to curved shapes
- Flood Fill - identifies and recolors connected pixel regions
- Bezier curves [7] - smooth parametric curves
- 2D geometric transformations - translation, rotation, scaling, and reflection for manipulating graphical objects

2.3 Application of Raster Graphics Editors

Raster editors are used in digital art, graphic design, and image editing. They have tools for freehand drawing, shape creation, color manipulation, and image filtering. From an educational standpoint, building a raster graphics editor gives you practical experience with the complete graphics pipeline, from user input and algorithm execution to rendering pixels on the display, making it a good project for understanding core computer graphics concepts.

2.4 Limitations and Challenges

Building a raster editor with Python and Pygame has some challenges:

- Python being interpreted can cause performance issues with lots of pixel-level operations
- Keeping the rendering responsive while processing drawing operations needs careful handling
- Pygame only gives basic graphical primitives and has no built-in GUI components
- Interface elements like toolbars, color pickers, and sliders must be built from scratch, adding to the development work

2.5 Development Approach

Pynter uses modular architecture in Python:

- Drawing tools are implemented as separate classes that can be switched at runtime
- Core algorithms (Bresenham's Line, Midpoint Circle/Ellipse, Flood Fill, Bezier curves) are implemented from their mathematical definitions rather than using library functions
- The Pygame library [8] provides low-level graphics and event handling that the application is built on

3. TOOLS AND TECHNOLOGIES

The application was built using cross-platform tools that support low-level graphics programming and real-time interaction.

Tool / Library	Version	Purpose
Python	3.13	Primary programming language
Pygame	2.6.1	Graphics engine and display management
OS	macOS / Linux / Windows	Cross-platform compatibility

Why Pygame:

- Provides direct pixel access using *surface.set_at()* and *surface.get_at()*
- Supports real-time rendering and event handling
- Simplifies window creation and graphics management
- Allows implementation of algorithms from scratch (no high-level drawing shortcuts)
- Pure Python → educational and easy to understand
- Cross-platform portability

4. SYSTEM ARCHITECTURE AND DESIGN

4.1 Overview Block Diagram

The application uses component-based architecture where each component handles its own event handling, rendering, and logic. The main window just coordinates everything.

The primary components are:

- **MainWindowGUI**: Controls the main event loop, manages rendering order, and processes global keyboard shortcuts
- **Canvas**: Provides an off-screen drawing surface and manages undo/redo functionality using snapshot storage
- **ToolSelect**: Implements the tool selection interface in the left sidebar, displaying 18 tools in a 2-column grid
- **ColorSelect**: Provides color selection through predefined swatches, an RGB slider picker, and action buttons (Save, Clear)

The components work independently and talk to each other through simple interfaces. The MainWindowGUI runs the frame loop:
Poll events → Dispatch events to components → Render components back-to-front → Update display

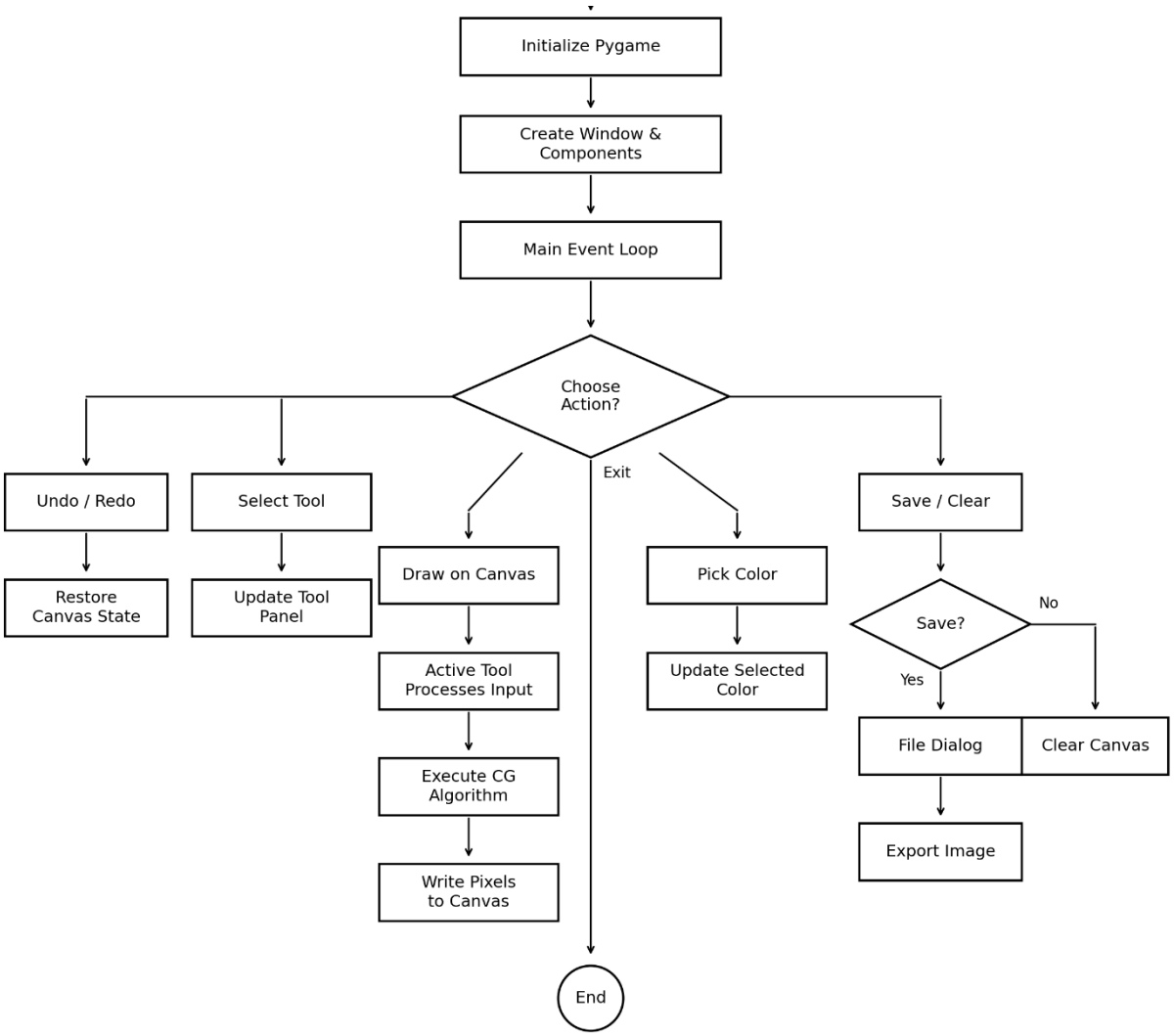

Fig 4-1: Overview Block Diagram

4.2 Component Architecture

Component	File	Responsibility
MainWindowGUI	main_window.py	Event loop, render order, shortcuts
Canvas	canvas.py	Drawing surface, undo/redo (50 max)
ToolSelect	tool_select.py	18 tools, 2-col grid, Strategy pattern
ColorSelect	color_select.py	24 colors, RGB picker, save/clear
Tool (ABC)	tools/tool.py	Interface: draw(), handle_events(), preview()
bitmap	bitmap.py	1-bit icons (24, 2× scale)

This modular structure makes it easy to add new tools or change existing ones.

Directory Structure:

```
└── anonyks-py_nter/
    ├── README.md
    ├── pynter.py
    └── pynter/
        ├── __init__.py
        ├── bitmap.py
        ├── canvas.py
        ├── color_select.py
        ├── globals.py
        ├── main_window.py
        ├── tool_select.py
        └── tools/
            ├── __init__.py
            ├── brush_tool.py
            ├── circle_tool.py
            ├── curve_tool.py
            ├── ellipse_tool.py
            ├── eraser_tool.py
            ├── eyedropper_tool.py
            ├── fill_tool.py
            ├── hypnotiser_tool.py
            ├── line_tool.py
            ├── magnifier_tool.py
            ├── mandala_tool.py
            ├── pencil_tool.py
            ├── rectangle_tool.py
            ├── select_box_tool.py
            ├── spiral_tool.py
            ├── square_tool.py
            ├── text_tool.py
            ├── tool.py
            └── triangle_tool.py
```

4.3 Rendering Pipeline

The rendering pipeline runs continuously inside the main frame loop at 120 FPS. It consists of four stages:

1. **Input Stage:**

- The system polls input events (mouse clicks, mouse movement, keyboard presses, window events) using the Pygame event system. Events are dispatched to the appropriate components - Canvas, ToolSelect, and ColorSelect.

2. **Processing Stage:**

- The currently active tool processes input events via its *handle_events()* method. The tool executes the appropriate graphics algorithm (Bresenham's Line, Midpoint Circle, Bezier Curve, Flood Fill, etc.). The resulting pixels are written on the off-screen canvas surface.

3. **Rendering Stage:**

- The system renders visual elements in back-to-front order:
  - Canvas surface
  - Tool preview overlay (temporary shape outlines)
  - Color selection panel
  - Tool selection panel
- This keeps the layers in the right order visually.

4. **Display Update Stage:**

- The final composed frame is displayed using *pygame.display.flip(),* and the frame rate is regulated using *clock.tick(120)* for smooth performance.

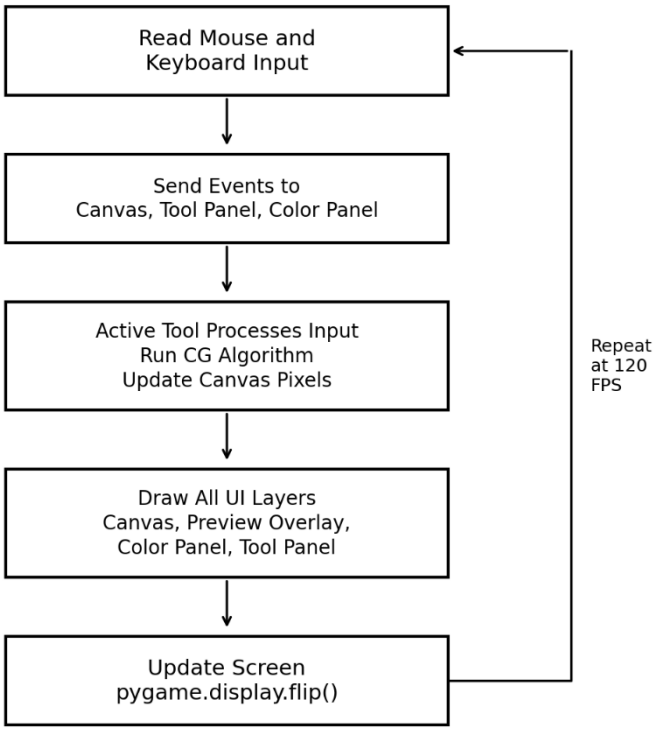

Fig 4-3: Rendering Pipeline Flowchart

5. ALGORITHM IMPLEMENTATION

This section goes through the graphics algorithms implemented in Pynter. Each drawing tool uses classical scan conversion algorithms implemented manually, no built-in graphics library shortcuts. This ties back to the rasterization and geometric transformation theory from the course.

5.1 Scan Conversion Algorithm

These algorithms convert continuous geometric shapes into discrete pixels on the raster grid.

5.1.1 Bresenham's Line Algorithm

File: tools/line_tool.py

Draws a straight line between two points using only integer arithmetic. The implementation handles all four slope cases by branching on whether the line is gentle or steep and whether the slope is positive or negative.

Algorithm Steps (gentle positive slope, |dy| ≤ |dx|):

1. Ensure left-to-right order: if x0 > x1, swap endpoints
2. Compute dx = x1 − x0, dy = y1 − y0
3. Set decision parameter p = 2*dy − dx
4. For x from x0 to x1:
   - Plot (x, y)
   - If p ≥ 0: y = y + 1, p = p + 2*dy − 2*dx
   - Else: p = p + 2*dy

Handling all octants: The algorithm branches into four cases based on the absolute values of dx and dy:

- Gentle positive slope (|dx| ≥ |dy|, dy > 0): step along x, increment y when p ≥ 0
- Gentle negative slope (|dx| ≥ |dy|, dy < 0): step along x, decrement y when p ≥ 0
- Steep positive slope (|dy| > |dx|, dy > 0): step along y, increment x when p ≥ 0
- Steep negative slope (|dy| > |dx|, dy < 0): step along y, decrement x when p ≥ 0

Time Complexity: O(max(dx, dy))

Space Complexity: O(1)

5.1.2 Midpoint Circle Algorithm

File: tools/circle_tool.py

Draws a circle using 8-way symmetry, calculates one octant and mirrors points.

Circle Equation: $\mathbf{x^2 + y^2 = r^2}$

Algorithm Steps:

1. Set x = 0, y = r, p = 1 − r
2. While x ≤ y:
   - Plot eight symmetric points:
     (cx + x, cy + y), (cx − x, cy + y), (cx + x, cy − y), (cx − x, cy − y),
     (cx + y, cy + x), (cx − y, cy + x), (cx + y, cy − x), (cx − y, cy − x)
   - x = x + 1
   - If p < 0: p = p + 2*x + 1
   - Else: y = y − 1, p = p + 2*(x − y) + 1

Time Complexity: O(r)

5.1.3 Midpoint Ellipse Algorithm

File: tools/ellipse_tool.py

Draws an ellipse using two regions based on slope.

Ellipse Equation: $\mathbf{x^2/rx^2 + y^2/ry^2 = 1}$

Algorithm Steps:

1. Set x = 0, y = ry, compute initial decision parameter p1
2. Region 1 (slope < 1): Plot four symmetric points (cx ± x, cy ± y), update decision parameter
3. Region 2 (slope ≥ 1): Plot four symmetric points (cx ± x, cy ± y), update decision parameter

Time Complexity: O(max(rx, ry))

5.2 Area Filling and Curves

5.2.1 Flood Fill Algorithm

File: tools/fill_tool.py

Fills a connected region from a seed point using BFS with 4-connectivity (up, down, left, right - no diagonals).

Algorithm Steps:

1. Record the target color at the seed pixel; if it already matches the fill color, return
2. Create a queue (deque) and enqueue the seed pixel; create a visited set
3. Lock the surface for fast pixel access
4. While queue is not empty:
   - Dequeue current pixel, set it to the fill color
   - For each of the 4 neighbors (right, left, down, up):
     - If the neighbor is within bounds, not visited, and matches the target color: mark visited and enqueue
5. Unlock the surface

The visited set prevents re-processing pixels, keeping the time complexity proportional to the filled area.

5.2.2 Bezier Curve Rendering

File: tools/curve_tool.py

Generates smooth curves using Bernstein polynomials. The user places an unlimited number of control points by clicking (quadratic, cubic, quartic, or higher degree) and can drag them to adjust the curve interactively before committing.

Bezier Equation:

$$\mathbf{B(t) = \Sigma\ [C(n,i) \times (1 - t)^{(n - i)} \times t^{i} \times P_i],\ t \in [0,1]}$$

where:

P_i are control points

C(n,i) is the binomial coefficient

n = number of control points − 1

Algorithm Steps:

1. For t from 0 to 1 (step size 0.0005, giving 2000 sample points):
   - Initialize x = 0, y = 0
   - For each control point P_i: compute Bernstein coefficient, accumulate x and y
   - Collect computed point
2. Connect all computed points using a polyline
3. When line width > 1, draw circles at endpoints to round off line caps

5.3 Transformations and Other Techniques

5.3.1 2D Geometric Transformations

Transformation	Equation for x′	Equation for y′
Rotation	$x' = x \cos\theta - y \sin\theta$	$y' = x \sin\theta + y \cos\theta$
Reflection (X-axis)	$x' = x$	$y' = -y$
Scaling	$x' = S_x \cdot x$	$y' = S_\gamma \cdot y$
Translation	$x' = x + T_x$	$y' = y + T_\gamma$

5.3.2 Mandala / Symmetry Drawing

File: tools/mandala_tool.py

Draws with N-fold radial symmetry around a center point. Each freehand stroke is duplicated N times by applying the 2D rotation transformation at evenly spaced angles.

Algorithm Steps:

1. User clicks to set center point (cx, cy) and begins drawing
2. For each stroke segment from point P1 to P2:
   - For i = 0 to N-1:
     - Compute angle $\theta = 2\pi \times i / N$
     - Translate point to origin: rel_x = x − cx, rel_y = y − cy
     - Apply rotation: x'=cx + rel_x cos θ − rel_y sin θ, y' = cy + rel_x sin θ+ rel_y cos θ
     - Plot rotated point

This is a direct application of the rotation formulas from §5.3.1, repeated N times to divide the full circle evenly.

5.3.3 Archimedes Spiral

File: tools/spiral_tool.py

Draws an Archimedes spiral outward from the center using polar-to-Cartesian conversion.

Spiral Equation: $\mathbf{r = a \times \theta}$

Where a = spacing / 2π controls the gap between successive arms.

Algorithm Steps:

1. Set center (cx, cy), compute max radius from drag distance
2. Starting from $\theta = 0$, step by 0.1 radians:
   - $r = a \times \theta$
   - $x = cx + r \cos \theta$
   - $y = cy + r \sin \theta$
   - Collect (x, y)
3. Connect all points to form the spiral curve

5.3.4 Monte Carlo Sampling (Spray Tool)

File: tools/brush_tool.py (spray mode)

Simulates spray painting by randomly distributing points inside a circular region using rejection sampling. Points outside the spray radius are rejected.

Algorithm Steps:

1. Repeat N times:
   - Generate random x, y inside bounding square
   - If $x^2 + y^2 \leq r^2$: plot (cx + x, cy + y)

5.3.5 Linear Interpolation for Opacity Blending (Eraser)

File: tools/eraser_tool.py

The eraser supports adjustable opacity (10% to 100%). Instead of just replacing pixels with the background color, it blends each pixel toward the background using linear interpolation (lerp) per color channel.

Blending Formula: **blended = original + (background - original) × t**

where t is the opacity factor (0.0 to 1.0). At t = 1.0 the pixel becomes fully the background color; at lower values it partially fades.

Algorithm Steps:

1. At the start of each stroke, save a copy of the entire canvas as the reference surface
2. For each pixel in the eraser region:
   - Read the original color from the saved reference (not the current canvas)
   - For each RGB channel: compute blended = original + (245 - original) × t
   - Write blended color using *set_at()*
3. At full opacity (100%), use a fast rectangle fill instead of per-pixel blending

6. WORKING PRINCIPLE

Pynter operates in an event-driven way - user interaction drives the entire workflow, with the graphics algorithms running behind each tool.

6.1 Setup

6.1.1 Initialization

When the application starts, the main module creates an instance of the MainWindowGUI class, which:

- Initializes the Pygame graphics engine
- Creates the main display window (1200 × 720 pixels)
- Instantiates core components: Canvas, ToolSelect, ColorSelect
- Initializes the frame clock at 120 FPS

6.1.2 Tool Selection

The user selects a drawing tool by clicking an icon in the left sidebar. This follows the Strategy Design Pattern, which means dynamic switching between drawing behaviors at runtime.

The process:

- ToolSelect detects the user's click
- It instantiates the selected tool subclass
- It assigns the new tool instance to *canvas.active_tool*

There are 18 tools arranged in a 2 × 9 grid, each with a custom 1-bit bitmap icon.

6.1.3 Color Selection

Preset Swatches: 24 predefined colors arranged in 2 rows × 12 columns in the top color panel.

Custom RGB Selection: Users can define custom colors using three sliders (Red: 0-255, Green: 0-255, Blue: 0-255). The selected color is stored globally and passed to the active tool during drawing.

6.2 Drawing Pipeline

6.2.1 Drawing Operation

When the user clicks or drags on the canvas, the drawing follows a structured pipeline:

Step 1: Snapshot Creation (Undo Support)
Before any drawing, the system saves the current canvas state to the undo stack (Memento pattern). This way the operation can be reversed.

Step 2: Algorithm Execution
The active tool executes its graphics algorithm:

- Line Tool → *_draw_line_bresenham()*
- Circle Tool → *_draw_circle_midpoint()*
- Ellipse Tool → *_draw_ellipse_midpoint()*
- Curve Tool → *_bezier_point()*
- Fill Tool → BFS Flood Fill
- Symmetry Tool → rotation and reflection transformations
- Spiral Tool → Archimedes polar equation
- Brush (spray mode) → Monte Carlo rejection sampling

Step 3: Pixel Rendering
Each algorithm writes pixels directly to the off-screen canvas surface using *surface.set_at((x, y), color),* direct low-level pixel manipulation.

6.2.2 Preview Rendering (Rubber-Band Effect)

During drag operations, before the mouse is released, the application shows a temporary preview of the shape being drawn (the "rubber-band effect"):

- Preview is drawn on the display surface only, does not modify the canvas
- Updates continuously at 120 FPS
- Provides real-time visual feedback (e.g., the Line Tool shows a temporary line from start point to current cursor)

6.3 State Management

6.3.1 Undo and Redo System

Uses the Memento pattern:

Undo (Ctrl + Z):

- Most recent snapshot removed from undo stack
- Current canvas state saved to redo stack
- Canvas restored to previous state

Redo (Ctrl + Y):

- Most recent redo snapshot restored
- Current state saved to undo stack

Memory Management:

- Maximum snapshots: 50
- Memory per snapshot: ~3 MB
- Maximum memory usage: ~150 MB

6.3.2 Saving the Canvas

Triggered by Ctrl + S:

- Converts the canvas surface to a PNG file using *pygame.image.save()*
- Saves as *my_painting.png* in the current working directory

6.3.3 Continuous Rendering Loop

The entire system runs inside a continuous loop at 120 FPS: process input events → update tool states → render previews → display updated graphics. This keeps drawing smooth and real-time.

7. IMPLEMENTATION DETAILS

7.1 Features and Tools

The application implements 18 drawing tools, each based on a specific graphics algorithm:

Tool	Algorithm Used	Description
Select Box	2D Transform (T, R, S, Reflect)	Select, move, rotate, scale, flip
Text	Font Rasterization	Keyboard text input
Eraser	Rectangle Fill	Erase with opacity (blend bg)
Fill	BFS Flood Fill	Fill connected region
Eye Dropper	Framebuffer Read	Sample color
Magnifier	2D Scaling	Zoom preview
Pencil	Point Rasterization	Fixed-width freehand
Brush	Circle / Monte Carlo	Variable-width, spray
Line	Bresenham Algorithm	Straight line
Curve	Bezier Curve	Smooth curve (control pts)
Square	Constrained Rectangle	Equal sides
Rectangle	Bounding Box	Axis-aligned rectangle
Circle	Midpoint Circle	Draw circle
Ellipse	Midpoint Ellipse	Draw ellipse
Triangle	Polygon Rasterization	Draw triangle
Spiral	Archimedes Spiral	Polar spiral
Hypnotizer	Animated Circle	Expanding rings

Each tool is a subclass of the Tool ABC, so adding new tools is straightforward.

7.2 Adjustable Tool Settings

Most tools have settings that can be adjusted in real-time using the scroll wheel or keyboard:

- Brush size: adjustable for Pencil, Brush, Eraser, and Symmetry tools (scroll wheel)
- Line width: adjustable for Line, Rectangle, Square, Circle, Ellipse, Triangle, and Spiral tools (scroll wheel)
- Eraser opacity: blends toward background color instead of hard erasing (Shift + Scroll)
- Spray density: controls how many dots per frame in spray mode (Shift + Scroll)
- Symmetry axes: number of radial copies in Mandala tool (scroll wheel, 3-16)
- Brush mode: toggle between circle and spray modes (Tab key)
- Magnifier zoom adjustable from 2x to 10x (scroll wheel)

7.3 1-Bit Bitmap Icon Rasterization

Tool icons use a custom 1-bit bitmap format:

- Resolution: 16 × 16 pixels
- Foreground pixel: #
- Transparent pixel: .
- Rendered at 2× scale (32 × 32 pixels)

Example (Line icon):

```
................
.............##.
............##..
...........##...
..........##....
.........##.....
........##......
.......##.......
......##........
.....##.........
....##..........
...##...........
..##............
.##.............
................
```

Each bitmap gets converted into a Pygame surface and shown as a tool icon. This gives us efficient rendering, barely any memory usage, and a retro look.

8. USER INTERFACE DESIGN

The UI follows a classic raster graphics editor layout with a retro look.

8.1 Color Scheme

The application uses a classic 3D beveled button style inspired by Windows 95/98:

Element	Color Name	Hex	RGB
Background / Canvas	Raywhite	#F5F5F5	(245,245,245)
Button Light	White	#FFFFFF	(255,255,255)
Button Shadow	Tan	#ACA899	(172,168,153)

This gives the buttons a raised/pressed 3D look like old Windows applications.

8.2 Layout Structure

The interface is divided into three main regions:

Color Panel (Top Bar): ~50 pixels high, contains 24 color swatches, RGB sliders, Save/Clear/Picker buttons.

Tool Panel (Left Sidebar): ~140 pixels wide, contains 18 tools in 2 columns × 9 rows with bitmap icons.

Canvas Area: Main drawing region (1200 × 720 pixels), white background, real-time drawing at 120 FPS.

8.3 1-Bit Bitmap Icons

All tool icons use custom 1-bit bitmaps:

- Lightweight and fast to render
- Consistent visual style
- Low memory usage
- Rendered dynamically at runtime

8.4 Key Controls

Input	Action
Left Click	Draw / select tool or color
Right Click	Preview brush size
Scroll Wheel	Adjust size / width / axes
Shift + Scroll	Adjust secondary settings
Tab	Switch brush mode
Ctrl + S	Save canvas
Ctrl + Z	Undo
Ctrl + Y	Redo
M	Toggle symmetry
Left Arrow	Previous color
Right Arrow	Next color
Enter	Commit

All controls are designed to be accessible without menus or dialogs, keeping the interaction fast and direct. The scroll wheel is used heavily since most tools have at least one adjustable parameter, and Shift + Scroll provides access to secondary settings without needing extra UI elements.

9. RESULT AND ANALYSIS

9.1 Initial Window

The main Pynter window with the tool panel on the left, color palette at the top, and the drawing canvas in the center.

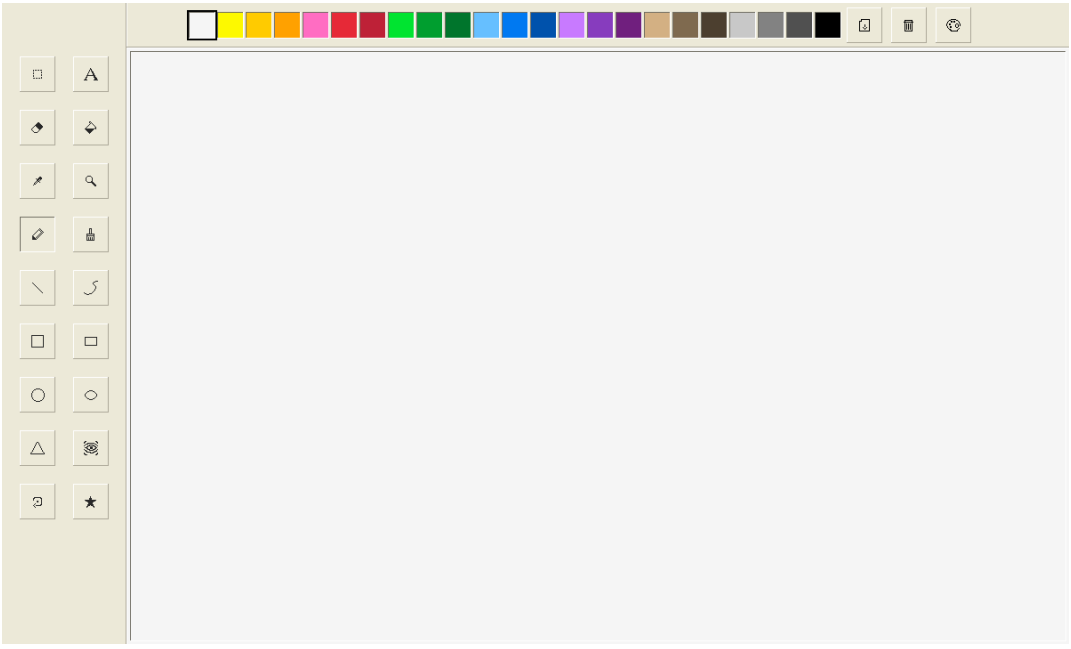

Fig 9-1: Main Application Window

9.2 Color Select

The 24 preset color palette and toolbar with Save, Clear, and RGB picker buttons.

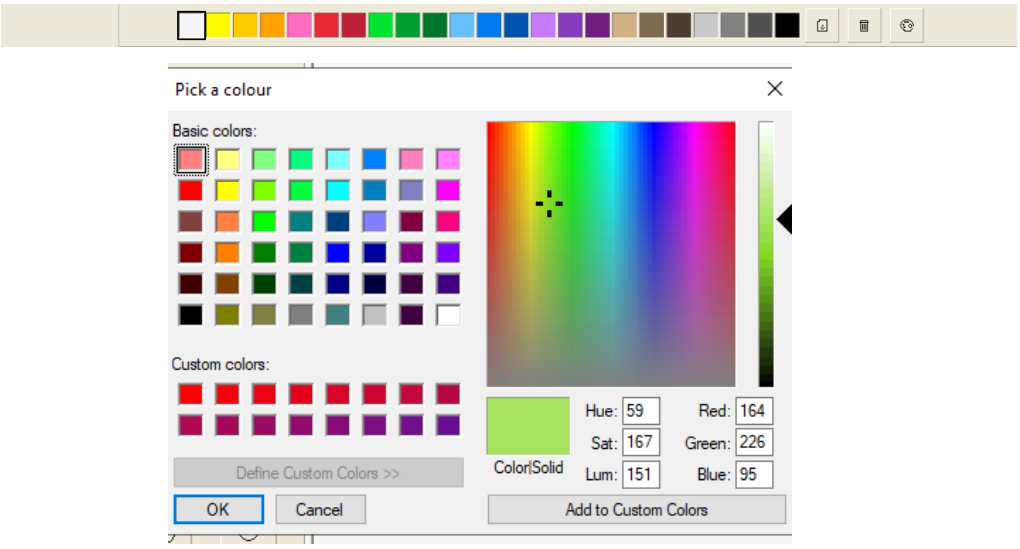

Fig 9-2: Color Select/Picker

9.3 Drawing Demo

A sunset landscape scene drawn entirely within Pynter - demonstrating rectangles, circles, polygons, and lines working together to create a full composition.

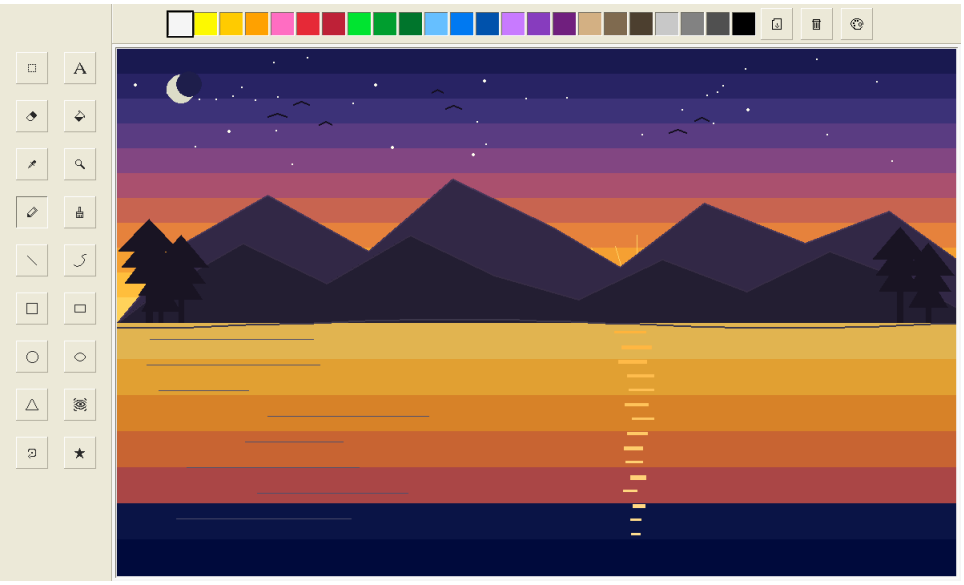

Fig 9-3: Drawing Demo [1]

"Celestial Bloom" - a mandala composition created using the Mandala, Hypnotizer, Spiral, Eraser (partial opacity), Brush (spray mode), and Bezier Curve tools on a dark canvas.

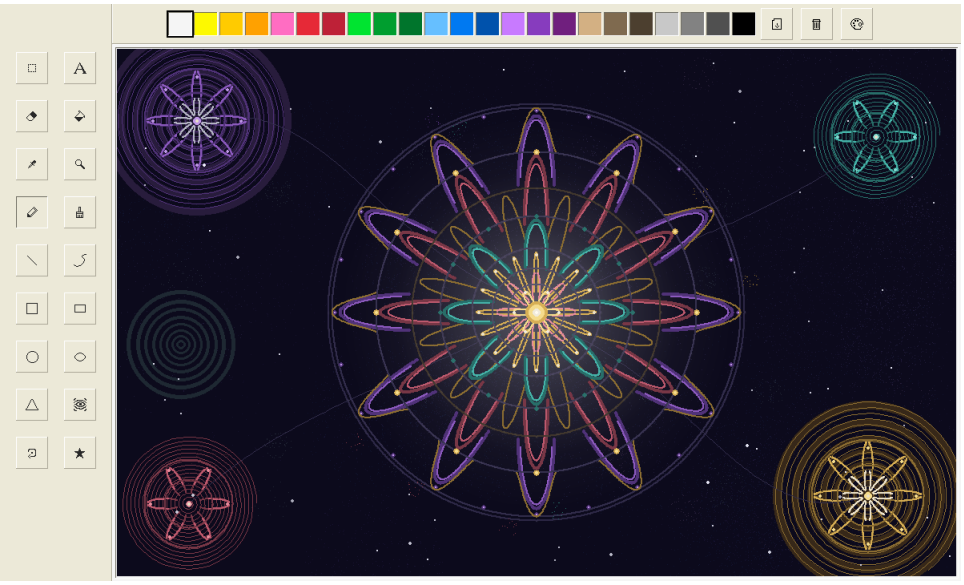

Fig 9-3: Drawing Demo [2]

- All 18 tools were tested and worked as expected

10. FUTURE ENHANCEMENTS

While Pynter covers the core algorithms from the course, there are several directions it could be extended in:

- **Layer Support:** Adding multi-layer editing with per-layer transparency and blending modes would let users work on different parts of a drawing independently. This would require managing multiple surfaces and composing them during the rendering stage.
- **Anti-Aliasing:** All shapes are drawn with hard pixel edges. Implementing Xiaolin Wu's line algorithm would produce smoother lines and curves by adjusting pixel intensity based on coverage.
- **Line Clipping:** Adding Cohen-Sutherland or Liang-Barsky line clipping would allow efficient handling of shapes that extend beyond the visible canvas area.
- **GPU Rendering:** Offloading rendering to the GPU using PyOpenGL or Moderngl could improve performance for larger canvases and complex operations.
- **Image Filters:** Adding operations like edge detection (Sobel, Laplacian), histogram equalization, and convolution-based blurring/sharpening would extend the project into basic image processing.
- **Flexible Canvas:** Supporting user-defined canvas dimensions with scrolling and panning would make the application more practical for different image sizes.
- **File Format Support:** Adding support for JPEG, BMP, and a custom project file format (to preserve undo history and tool states) would make workflows more flexible.
- **3D Extensions:** Extending with basic 3D wireframe rendering, projection (orthographic/perspective), and hidden surface removal would cover more topics from the computer graphics curriculum.

11. CONCLUSION

Pynter shows that the core computer graphics algorithms we studied in class can be put into practice in an interactive raster editor. The project implements Bresenham's Line, Midpoint Circle/Ellipse, 2D transformations, Bezier curves, Flood Fill, and Monte Carlo sampling across 18 drawing tools.

Key strengths:

- All algorithms are implemented from scratch using integer arithmetic, not library shortcuts, which ties directly to computer graphics theory
- Uses Strategy pattern for tools, ABC for interfaces, and Memento for undo/redo
- Runs at 120 FPS, supports up to 50 undo/redo levels, 24 preset colors + RGB picker, and works on Windows, macOS, and Linux
- Total codebase: 26 Python files, ~3,850 lines

12. APPENDICES

Appendix A: Project Timeline

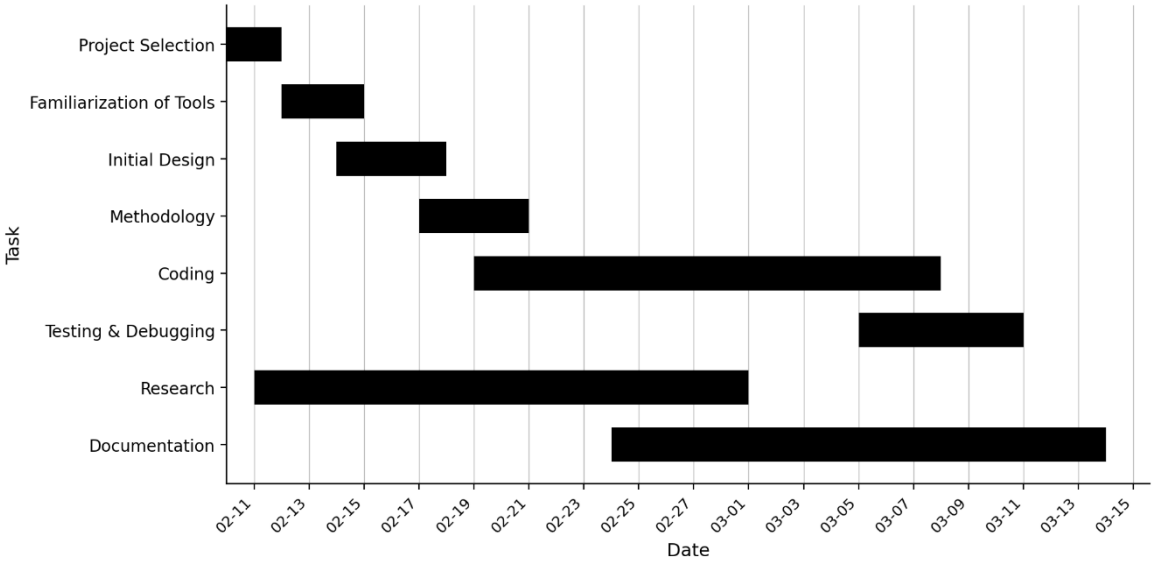

Fig 12-1: Gantt Chart

References

[1] D. Hearn and M. P. Baker, *Computer Graphics C Version*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1997.

[2] Pygame Community, “Pygame Documentation”, *pygame.org*. [Online]. Available: https://www.pygame.org/docs/. [Accessed: Mar. 2026].

[3] B. Unutmaz, "Pyint Pixel-Painter," GitHub. [Online]. Available: https://github.com/burakunutmaz/Pyint_Pixel-Painter. [Accessed: Mar. 2026].

[4] Fiesty-Cushion, "Pixel-Craft," GitHub. [Online]. Available: https://github.com/FiestyCushion/Pixel-Craft. [Accessed: Mar. 2026].

[5] MyreMyalar, "Made a basic paint program with Pygame and pygame_gui," Reddit r/pygame, Apr. 2020. [Online]. Available: https://www.reddit.com/r/pygame/comments/fzzqr3/made_a_basic_paint_program_with_pygame_and_pygame. [Accessed: Mar. 2026].

[6] J. E. Bresenham, "Algorithm for computer control of a digital plotter," *IBM Syst. J.,* vol. 4, no. 1, pp. 25-30, 1965. doi: 10.1147/sj.41.0025

[7] Freya Holmer, "The Beauty of Bezier Curves," YouTube, 2021. [Online]. Available: https://youtube.com/watch?v=aVwxzDHniEw. [Accessed: Mar. 2026].

[8] Pygame Community, "pygame," GitHub. [Online]. Available: https://github.com/pygame/pygame. [Accessed: Mar. 2026].